



نظري

7

12

1440 sp

كلية الهندسة المعلوماتية

السنة الثانية

compilation والعمليات الأساسية (التوابع العودية)

د. سيرا أستور



RB Informatcs; 26/05/2025

بنیان الحاسوب ا

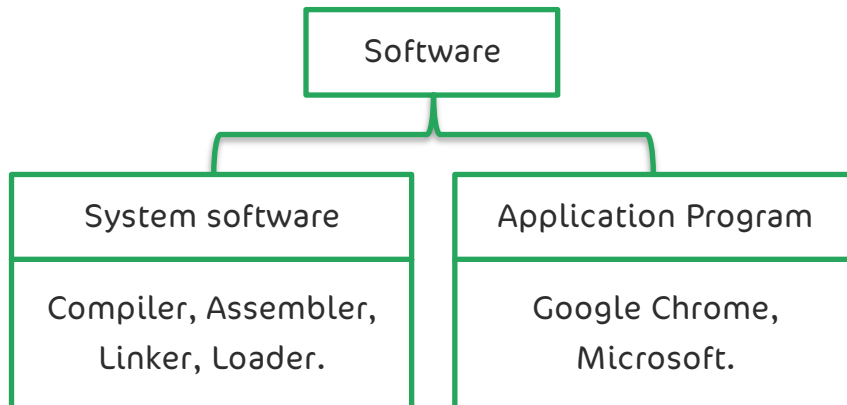
بسم الله الرحمن الرحيم

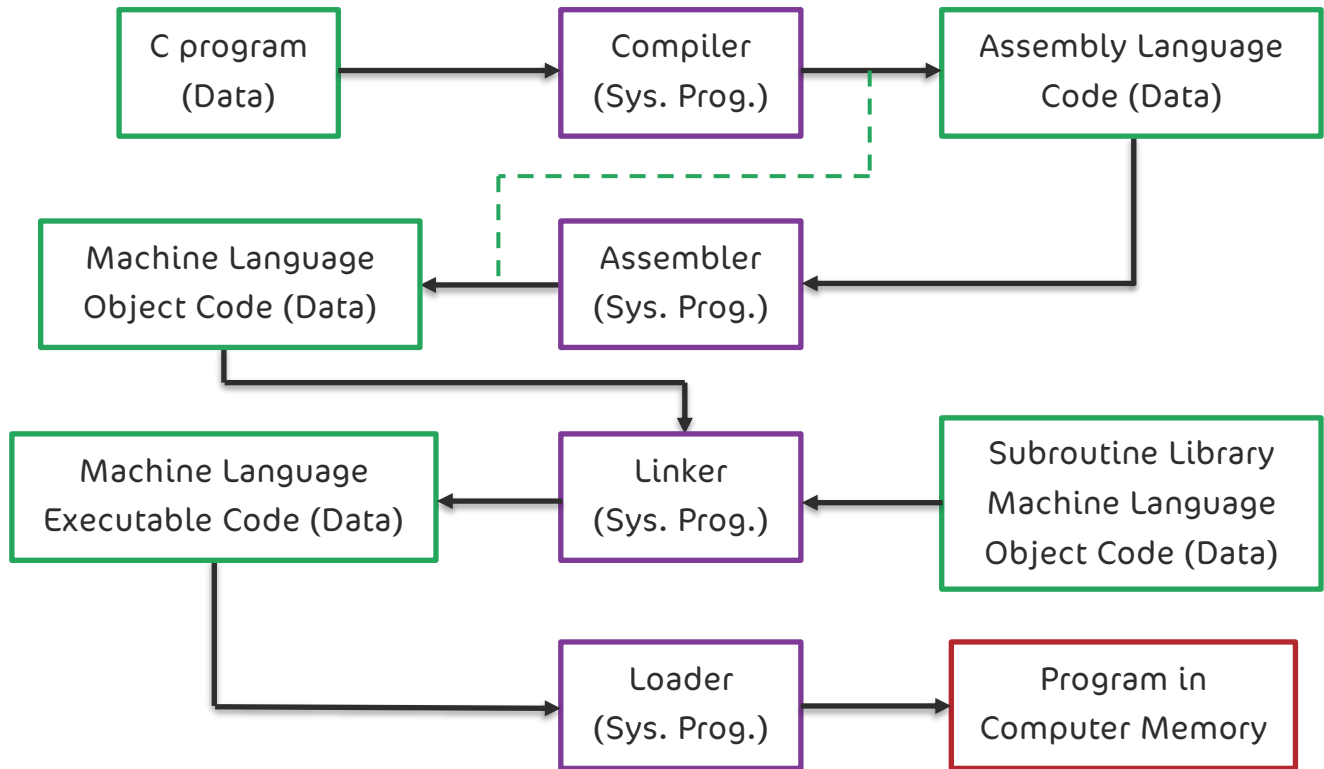
مراحل تحويل برنامج إلى برنامج يعمل على الحاسوب:

1. C program (data): كود بلغة عالية المستوى.
2. Compiler: يحول الكود من لغة عالية المستوى إلى لغة المجمع (Assembly) التجميع.
3. Assembler: يحول لغة التجميع إلى لغة الأرقام والواحدات لغة الآلة.
4. Linker: يدمج جميع ملفات الكود مع المكتبات لينتج ملف تنفيذي كامل.
5. Loader: يحمل البرنامج من Hard disk إلى ذاكرة الحاسوب ليصبح جاهزاً للتنفيذ.
6. التنفيذ: الحاسوب يشغل البرنامج في ذاكرته وينفذ الأوامر.

ملاحظة:

يختلف المكان في الذاكرة في كل استدعاء جديد للبرنامج.
الفرق بين **object code** و **Executable code** أن:
object code هو الشيفرة التي تنتج بعد ترجمة الكود إلى رموز مفهومة للآلة ولكنه غير جاهز للتنفيذ لأنه قد يحتاج إلى ربط البرنامج.
أما **Executable code**: الشيفرة أصبحت جاهزة للتشغيل مباشرة على الحاسوب لأنها تضم كل ما يلزم من تعليمات وبيانات بعد عملية الربط (linking) والتحميل (loading).





Compiler:

Inputs:

- كود البرنامج المكتوب بـ HLL مثل C++.
- مجموعة التعليمات الخاصة بمعمارية الحاسوب (ربما تشمل تعليمات زائفة).
- تنظيم الذاكرة والسجلات (Registers) في المعالج.

Outputs:

- كود بلغة التجميع (Assembly Language).

وظائف Compiler الخاصة بمعمارية MIPS تشمل:

- تكون متخصصة بناءً على مجموعة التعليمات (ISA) وتنظيم الجهاز.
- يخصص المتغيرات في سجلات المعالج.
- يترجم لغة البرمجة إلى لغة التجميع.
- HLL <-- assembly Languages (Machine code)
- عند ترجمة دالة فرعية (subroutine) أو إجراء (procedure) يستعمل السجلات \$s0 إلى \$s7 يقوم بحفظ محتوياتها ثم استعادتها لاحقاً للحفاظ على قيمها.

■ مثال أرسيزي للتوضيح:

Code C++:

```
int sum (int a, int b) {
    return a+b;
}
```

الترجمة إلى لغة التجميع:

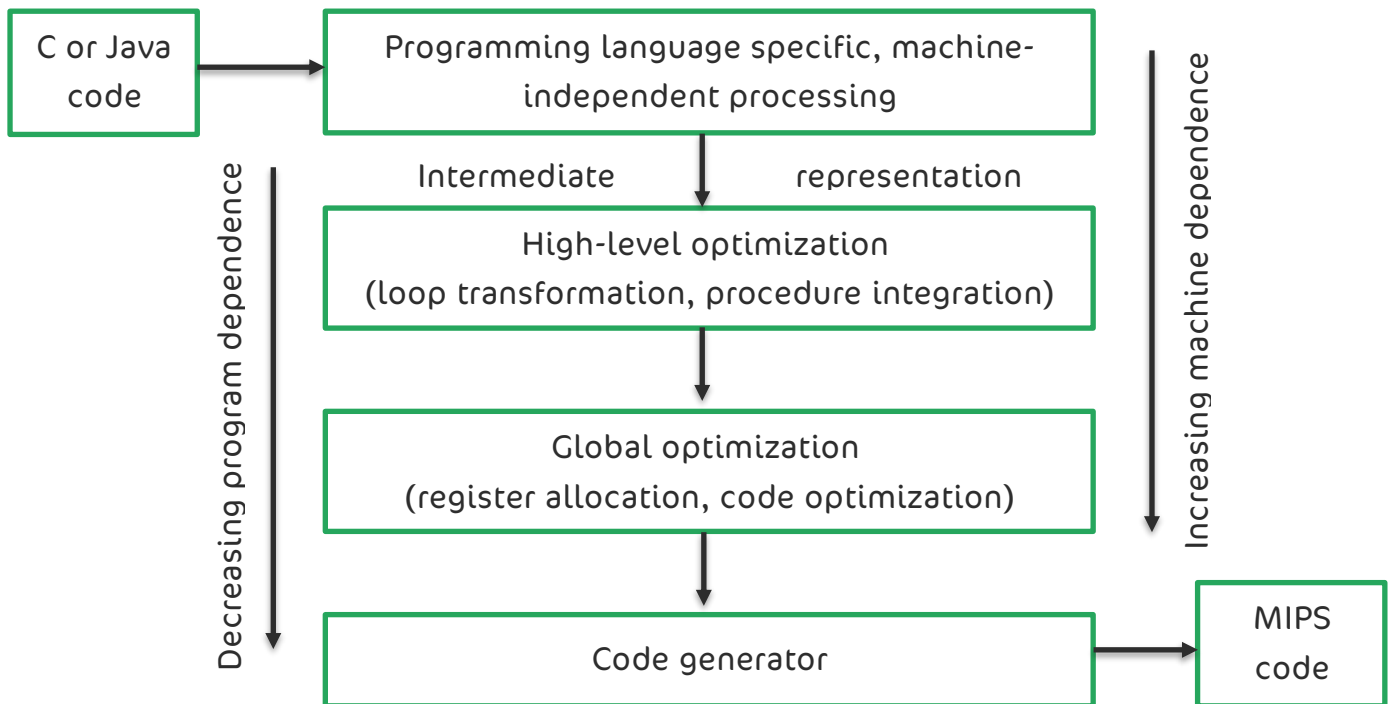
Assembly code:

Sum:

```
add $s0, $a0, $a1 # $s0=a+b
add $v0, $a0, $a1
jr $ra
```

- نجمع المتغيرين a, b الموجودين في \$a0, \$a1 ونضع النتيجة في \$r0 (مكان ناتج الدالة).
- نرجع إلى نقطة استدعاء الدالة (return).

مراحل الترجمة والتحسين في عملية تحويل الكود من لغات عالية المستوى إلى كود MIPS:



نبدأ بكتابة برنامج بلغة عالية المستوى ثم نمر بمراحل تحسين وتحليل تدريجية لكنها عامة ومخصصة للغة فقط، بعد ذلك ننتقل إلى تحسينات أكثر اعتماداً على نوع المعالج، ونصل أخيراً إلى توليد كود قابل للتنفيذ على معمارية MIPS.

■ ملاحظة أرسيزية:

ما المقصود بمعمارية MIPS:

هي اختصار لـ Microprocessor without Interlocked pipeline stages.

وهي نوع من معمارية الحواسيب تعتمد على تصميم بسيط وفعال لوحدة المعالجة.

وبشكل أبسط: معمارية MIPS هي مجموعة قواعد وتعليمات (Architecture-1st Instruction Set) تحدد كيف يتعامل معالج الكمبيوتر مع الأوامر والبيانات داخل النظام.

Compiler Optimization

مفهوم: يعني جعل الكود الناتج يحتوي على أقل عدد ممكن من تعليمات المعالج وهذا لتحسين الأداء والسرعة وتقليل استهلاك الموارد.

■ مثال:

$$X[i] = X[i] + 4$$

بدل تكرار عنوان $X[i]$ أكثر من مرة يتم حساب العنوان مرة واحدة فقط، حفظه في سجل (Register) ثم استخدامه في تعليمات التحميل (lw) والتخزين (sw) هذا ما يسمى التخلص من التعبيرات المشتركة (common sub-expression elimination).

- التحسين قد يتم محلياً داخل كتلة كود واحد (local optimization) أو عبر عدة كتل (global optimization).
- تحسين توزيع السجلات (global register allocation) توزيع أفضل للسجلات لتقليل استخدام الذاكرة.
- تقليل القوة (strength reduction): بدلاً من استخدام الضرب المتكرر (مكلف) يمكن استبداله بتعليمات أسرع مثل الإزاحة المنطقية لليسار (shift Logical Left) مثلاً 2^k يتم عبر إزاحة البتات بدلاً من عملية الضرب.
- إزالة التعليمات غير الضرورية: إذا كان هناك قيمة لن تستخدم لاحقاً يمكن عدم تخزينها في الذاكرة (إزالة تعليمات sw)، لأن compiler يقوم بقراءة الكود كامل على عكس interpreter الذي يقرأ الكود سطر سطر.

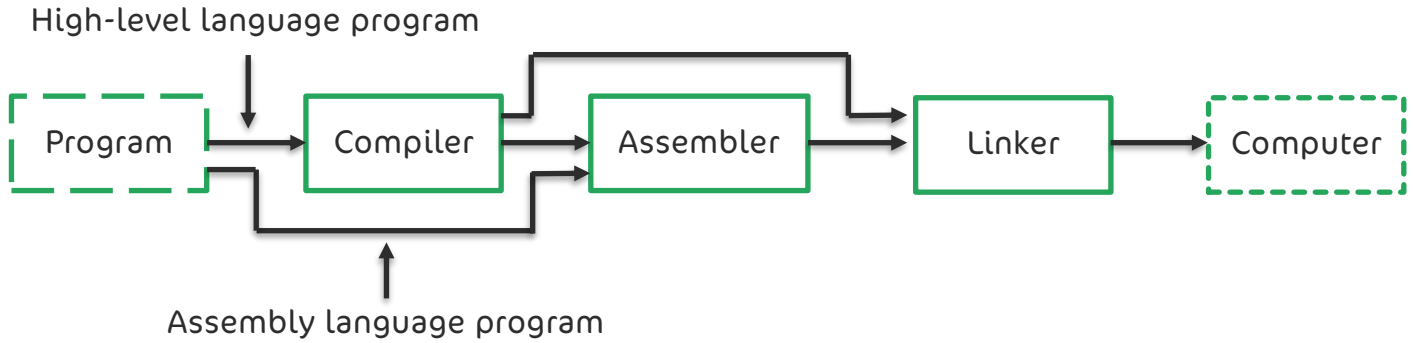
Assembly Language

دورها:

■ تكون الخرج الناتج عن عمل Compiler.

■ كما تستخدم لغة التجميع لكتابة البرامج عندما تكون سرعة تنفيذ البرنامج أو حجمه أمراً مهماً للغاية أي عندما لا يكون هناك بديل جيد من اللغات عالية المستوى وتحتاج البرامج إلى أداء عالي جداً أو حجم صغير جداً.

بمعنى مختصر: لغة التجميع تعطي تحكم دقيق في كيفية عمل البرنامج على الجهاز.



ما هو الـ Assembler:

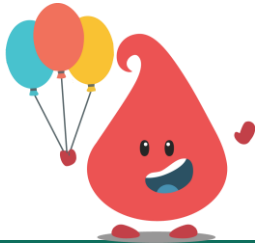
هو (system program) برنامج نظام يقوم بترجمة لغة Assembly إلى لغة الآلة.

دوره:

- يقوم بتحويل برنامج لغة التجميع إلى (object file) الذي يحتوي على تعليمات لغة الآلة والبيانات والمعلومات اللازمة لتحديد مواقع التعليمات بشكل صحيح في الذاكرة.
- يقوم بتحويل التعليمات الزائفة (pseudo Instructions) إلى تعليمات أساسية باستخدام السجل \$at.
- لتوليد النسخة الثنائية Binary Version لكل تعليمة، يجب على الـ Assembler تحديد العناوين المتوافقة مع جميع العلامات التوجيهية وتعليمات نقل البيانات لذا ينشئ جدول رموز (Symbol table) ومعلومات إعادة التوجيه (Relocation Information).

كيف يقوم Assembler بترجمة البرنامج؟

1. يبحث عن كل العلامات (labels) ويجمع معلومات عنها.
 2. يبدأ بإنتاج التعليمات الفعلية (Machine code) باستخدام معلومات جدول الرموز (Symbol table).
- أثناء المرور الثاني (Second pass) يستخدم جدول الرموز لإنتاج الكود النهائي.
 - لكن إذا كان من الضروري زيادة سرعة الـ Assembler يمكن تنفيذ هذه العمليات في مرور واحد باستخدام تقنية Back patching.



■ شرح أرسيزي:

:Back-patching

يقوم الـ Assembler بملء العناوين المفقودة لاحقاً عندما تتوفر المعلومات بدلاً من الانتظار لتمرير إضافي.

Symbols & Symbols table

- Symbols: هي أسماء برامج أو إجراءات (procedures) أو تسميات (Labels) داخل البرنامج.
- في الشفرة التنفيذية (executable code) كل رمز يشير إلى عنوان في الذاكرة.
- في ملف الكائن (object code) القيمة النهائية للرمز (أي العنوان) قد تكون غير محددة بعد.

رموز خارجية (External Symbols):

مثل أسماء الإجراءات الفرعية (subroutine names) أو إجراءات المكتبة (Library procedures) التي لم تحدد عناوينها بعد (unresolved).

رموز داخلية (Internal Symbols):

مثل تسميات الجمل البرمجية (statement labels) التي لها عناوين نسبية داخل البرنامج.

■ ملاحظة:

في هذه المرحلة القيمة النهائية للرمز تكون غير محددة، تحدد خلال مرحلة Loader سنتعرف عليه خلال المحاضرة.

Symbols Table:

هو جدول يطابق أسماء العلامات (الرموز) مع عناوين الكلمات في الذاكرة التي تمثلها الأوامر البرمجية، كما يستخدم لمساعدة Assembler أو Linker في تحديد أماكن تعليمات البرنامج.

Relocation Information:

تستخدم لتحديد التعليمات والبيانات التي تعتمد على عناوين مطلقة، هذه العناوين يجب تعديلها إذا تم نقل أجزاء من البرنامج إلى مواقع مختلفة في الذاكرة.

■ ملاحظة:

العناوين المطلقة المجردة:

هي العناوين التي لا تحتاج إلى ضرب أو إزاحة مثل عناوين الأجهزة (Hardware Addresses) تكون ثابتة في النظام.

عناوين في الذاكرة ثابتة (ROM Addresses):

مثل عناوين البرامج والبيانات المخزنة في الذاكرة الدائمة.

Linker:

مفهوم: هو برنامج نظام يقوم بدمج لغة الآلة التي تم تجميعها بشكل مستقل.

assembled machine language programs.

مسؤوليته الرئيسية هي حل جميع الرموز الغير معرفة (Undefined Labels) وتحويلها إلى ملف تنفيذي (executable File).

كيف يعمل الـ Linker:

يستخدم معلومات إعادة التوجيه (relocation information) و جدول الرموز (symbol table) من كل وحدة كائن (object module) لحل الرموز الغير معرفة.

Input:

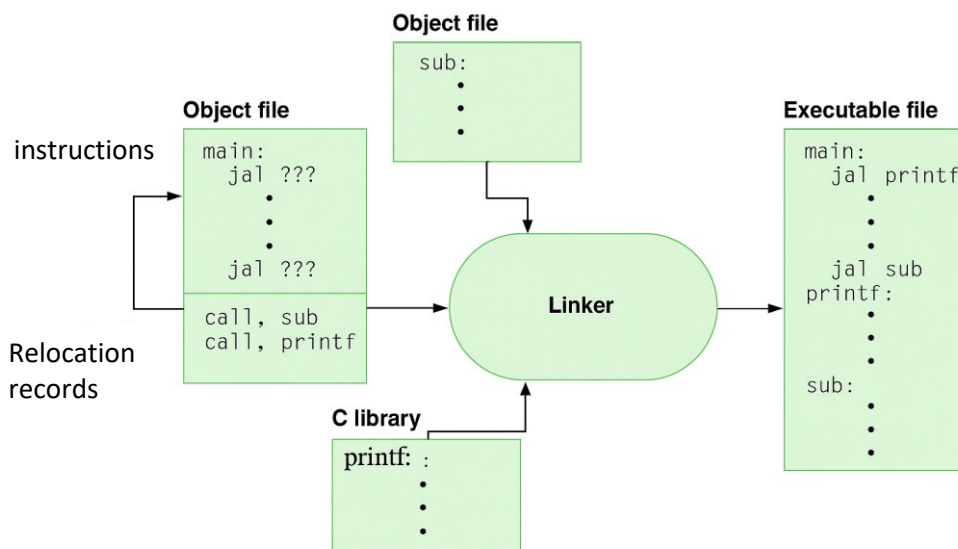
مكتبات وبرامج وإجراءات كلها بصيغة تعليمات لغة الآلة.

Output:

ملف تنفيذي بصيغة تعليمات لغة آلة كذلك لكن بدون أي مراجع غير محددة (reference) لبيانات من مكتبة أو إجراء خارجي) أو معلومات إعادة التوجيه.

المهام الثلاثة الرئيسية لـ Linker:

1. تحديد مواقع الذاكرة:
يحدد أماكن ذاكرة كل كود من الوحدات البرمجية المختلفة ويقوم بتعديل التعليمات التي تحتوي على عناوين مطلقة absolute references لتناسب المواقع الجديدة في الذاكرة.
2. حل المراجع بين الملفات:
يربط التعريفات والرموز بين ملفات الكائن المختلفة لضمان أن كل استدعاء لرمز معين يشير للمكان الصحيح.
3. البحث في مكتبات البرامج:
يبحث في مكتبات البرامج ليجد روتينات المكتبة التي يحتاجها البرنامج ويربطها.



■ بالنتيجة:

كل الأجزاء ترتبط بشكل صحيح مع العناوين المناسبة في الذاكرة والبرنامج يعمل كما يجب.

المصطلح	التعريف	النطاق والاستخدام
روتين (Routine)	مصطلح يشير إلى أي كتلة برمجية قابلة لإعادة الاستخدام والاستدعاء، يشمل الدوال، الإجراءات، التوابع، إلخ....	مصطلح غير مرتبط بلغة محددة، ويستخدم غالباً في البرمجة منخفضة المستوى، أو في التوثيق التقني.
دالة (function)	نوع محدد من الروتينات، وهو عبارة عن روتين يرجع قيمة عند التنفيذ.	شائع في لغات البرمجة الحديثة مثل C، Javascript، Python، إلخ....

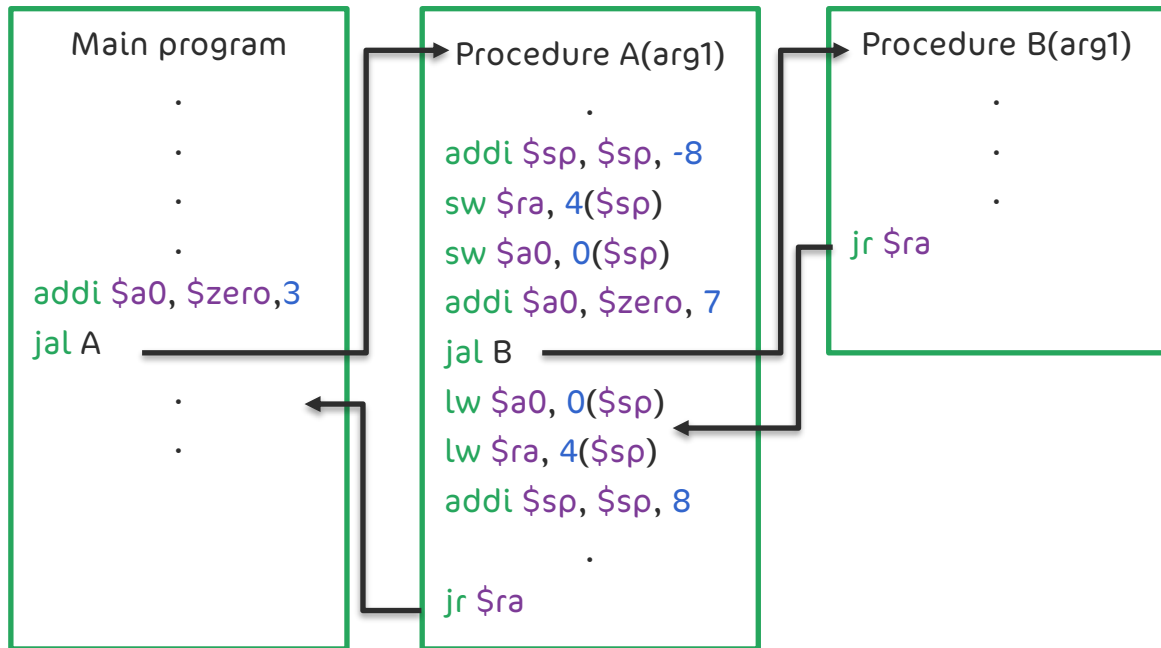
Loader:

مفهوم:

هو برنامج مسؤول عن تجهيز البرنامج ليعمل في الذاكرة، حيث يقوم بقراءة ملف البرنامج التنفيذي (machine code والبيانات) من القرص ويحضرها في الذاكرة ليتم تنفيذها.

خطوات عمل الـ Loader في Unix:

1. قراءة رأس الملف التنفيذي (Executable Header):
يحدد حجم أقسام التعليمات (Text) و البيانات (Data) لكي يعرف كم من الذاكرة يحتاج.
2. إنشاء مساحة عناوين مناسبة:
يقوم بفتح منطقة في الذاكرة كبيرة كفاية لاستيعاب تعليمات البرنامج و بياناته.
3. نسخ التعليمات والبيانات إلى الذاكرة:
ينقل محتويات الملف التنفيذي (الكود و البيانات) إلى هذه المساحات في الذاكرة.
4. نسخ معطيات الإدخال (parameters) إلى stack البرنامج الرئيسي:
إذا كان البرنامج يحتوي على معطيات تمرر له يقوم بإضافتها إلى Stack.
5. تهيئة مسجلات الجهاز وضبط مؤشر stack:
يضبط المسجلات (Registers) في المعالج ويحدد مؤشر المكس (Stack Pointer) على أول عنوان خالي في الذاكرة.
6. تشغيل روتين البداية (startup Routine):
تقوم هذه الخطوة بنقل المعطيات من المكس للسجلات المناسبة، ثم تستدعي الدالة الرئيسية main بالبرنامج عند انتهاء main، روتين البداية ينهي البرنامج عبر استدعاء نظام الخروج.



ماذا يحدث في الذاكرة؟

عندما يقوم تابع باستدعاء تابع آخر يجب تنظيم بعض المعلومات في الذاكرة لكي يدير هذا الاستدعاء ترتيبات العودة بشكل صحيح تخزن ضمن إطار الاستدعاء (Stack Frame).

ما هو إطار الاستدعاء (stack frame)؟

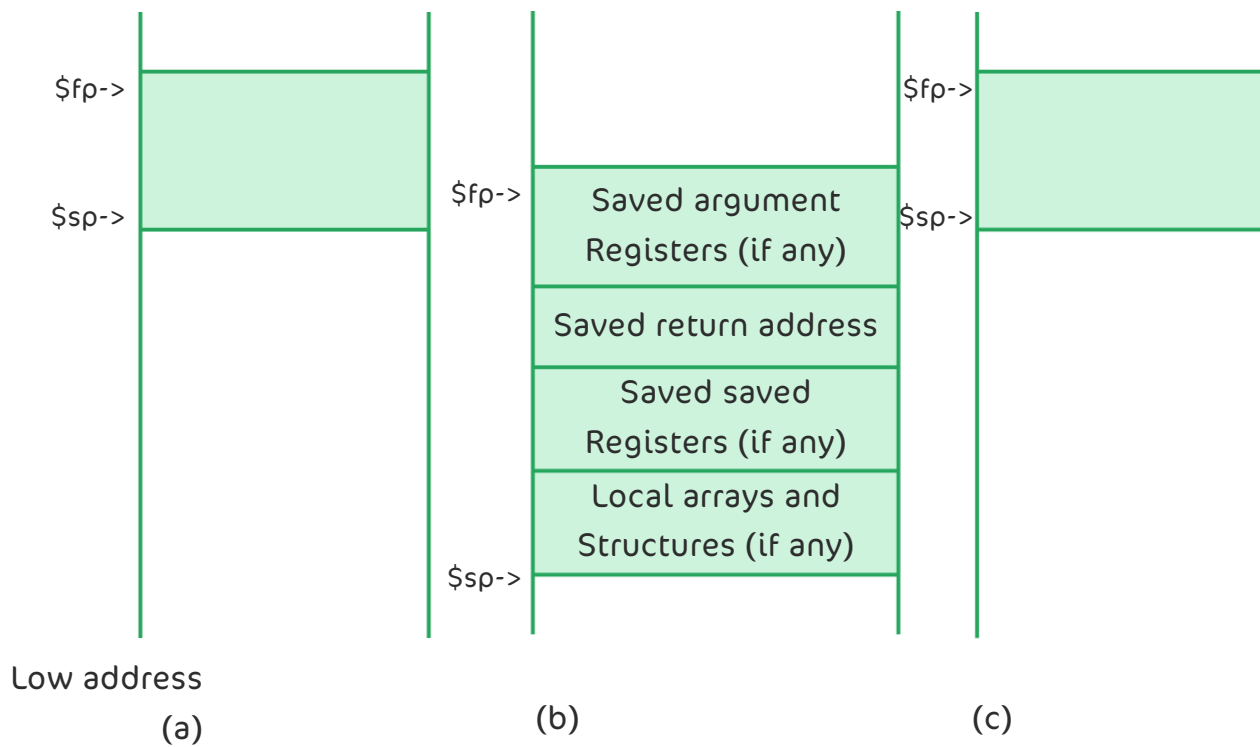
هو مساحة في الذاكرة تستخدم لتنظيم وحفظ عدة أشياء مهمة أثناء تنفيذ الاجراء بحيث يساعد ذلك على:

1. تمرير القيم Arguments إلى الإجراء، أي القيم التي نمررها كمدخلات إلى الإجراء يتم وضعها داخل إطار الاستدعاء لكي يتمكن الإجراء من الوصول إليها.
2. حفظ السجلات (Registers):
بعض السجلات في المعالج قد يتم تعديلها داخل الإجراء، وإذا كان التابع يريد أن يحافظ على قيم هذه السجلات دون تغيير، حيث يتم حفظ هذه القيم في إطار الاستدعاء مؤقتاً.
3. توفير مساحة للمتغيرات المحلية للإجراء:
إذا كان للإجراء متغيرات خاصة به، فهذه المتغيرات تخزن داخل هذا الإطار في الذاكرة.



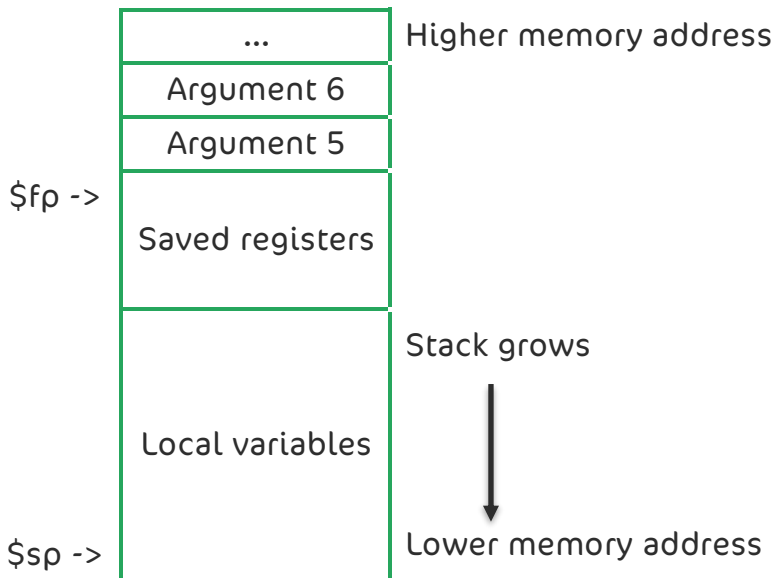
تذكرة:

شكل مساحة البيانات الجديدة في stack:



أول 4 معطيات يتم تمريرها عبر السجلات في المعالج هو ما يجعل الوصول إليها أسرع و أكثر كفاءة.

الخامس وما بعده يتم وضع المعطيات الإضافية على المكس (stack) داخل إطار الاستدعاء (stack frame).



Example:

```
int fact(n) {
    if (n < 1) return (1);
    else return (n * fact (n-1));
}
```

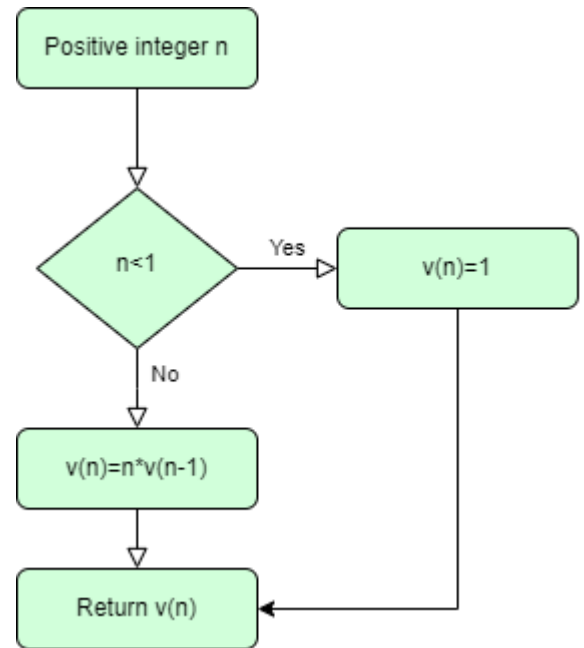
فكرة البرنامج:

البرنامج يحسب العامل للعدد الصحيح الموجب.

اسم التابع Fact:

إذا كان $n < 1$ يعيد (1) (حالة الحد لأن $0! = 1$)

وإلا يعيد $n * \text{Fact}(n-1)$.



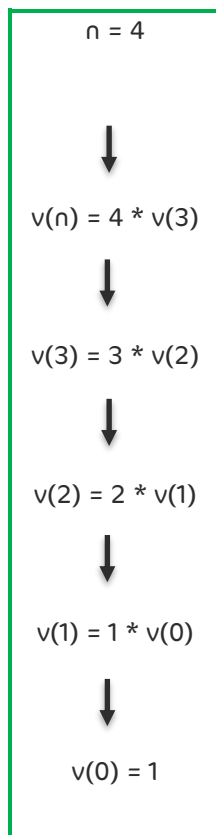
Main program calls fact, \$a0 = 4

fact calls fact, \$a0 = 3

fact calls fact, \$a0 = 2

fact calls fact, \$a0 = 1

fact calls fact, \$a0 = 0



Returns \$v0 = \$v0 * \$a0 = 24

Returns \$v0 = \$v0 * \$a0 = 6

Returns \$v0 = \$v0 * \$a0 = 2

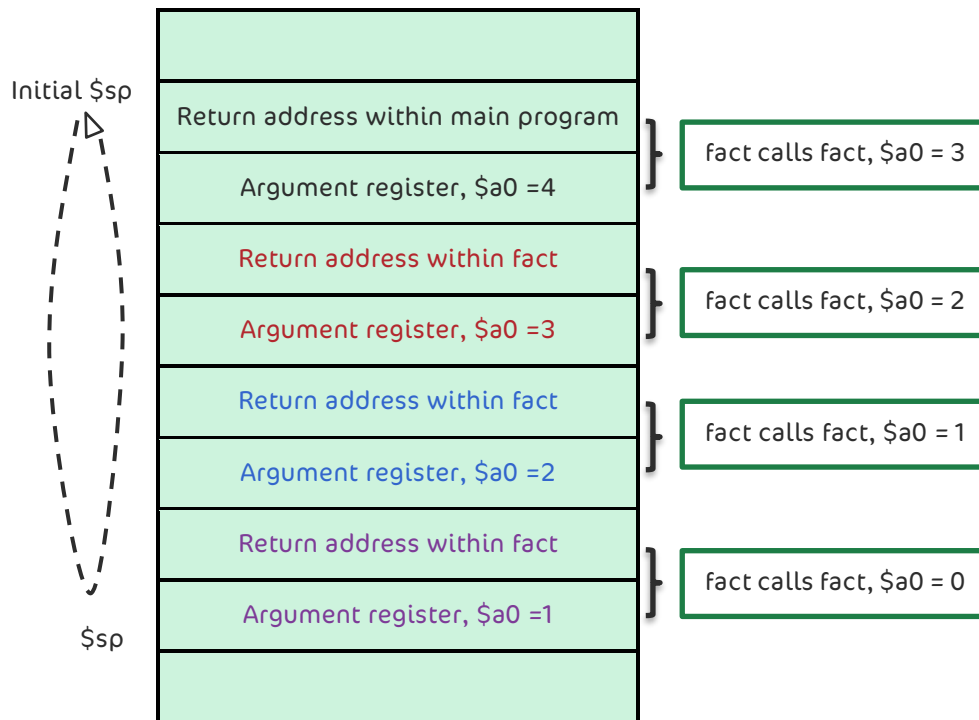
Returns \$v0 = \$v0 * \$a0 = 1

Returns \$v0 = 1

عند التنفيذ بلغة التجميع:

الاستدعاء الذاتي
:(Recursion)

بما أن الإجراء يستدعي نفسه
يجب إدارة كل استدعاء بشكل
مستقل.



إدارة المكس stack:

لحفظ معلومات الاستدعاءات السابقة مثل عنوان العودة (Return Address) وقيمة المتغيرات (مثل n) قبل استدعاء الإجراء مجدداً. حفظ و استرجاع السجلات المهمة خاصة سجلات ra (return address) و a0 الذي يحمل المعطى n.

```

1 fact:
2     addi $sp, $sp, -8      # adjust stack for two items
3     sw $ra, 4($sp)        # save return address of caller
4     sw $a0, 0($sp)        # save caller supplied argument n
5
6     slti $t0, $a0, 1      # $t0 = 1, if n < 1, i.e., n = 0
7     beq $t0, $zero, L1    # go to L1, if n ≥ 1
8     addi $v0, $zero, 1    # return $v0 = 1 to caller
9     addi $sp, $sp, 8      # No need to restore registers,
10                                # since none was changed, but
11                                # must restore stack pointer
12     jr $ra                # return to caller instruction after jal
13
14 L1:
15     addi $a0, $a0, -1     # set $a0 to n-1
16     jal fact              # call fact with argument n-1
17     lw $a0, 0($sp)        # on return from fact, restore n
18     lw $ra, 4($sp)        # restore return address
19     addi $sp, $sp, 8      # adjust stack pointer
20
21     mul $v0, $a0, $v0     # n! = n * (n-1)!
22     jr $ra                # return to caller
    
```

الخطوات الأساسية للكود:

1. إعداد stack frame:
 - تخصيص مساحة في المكس (عادة 8 أو 32 بايت حسب الحاجة).
 - حفظ عنوان العودة \$ra وقيمة المتغير \$a0 في المكس.
 - تعديل مؤشر المكس \$sp.
2. التحقق من شرط الحد:
 - إذا كان $n < 1$ استخدام `beq` و `slti` للتحقق، يتم إرجاع القيمة مباشرة.
3. الاستدعاء الذاتي:
 - تقليل n بمقدار واحد (`addi $a0, $a0, -1`) استدعاء الدالة `fact` باستخدام `jal fact`.
4. استرجاع القيم:
 - بعد العودة من الاستدعاء الذاتي يتم تحميل n القديم و عنوان العودة من المكس.
 - تعديل مؤشر المكس لإزالة البيانات المحفوظة.
5. حساب العامل:
 - ضرب n في القيمة التي أعيدت من الاستدعاء الذاتي `mul $v0, $a0, $v0`
6. العودة إلى المستدعي باستخدام `jr $ra`.



للاستزادة يمكن قراءة بقية السلايدات، بالإضافة للاطلاع للمرجع.
انتهت المحاضرة..
بالتوفيق!!